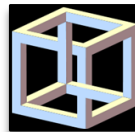


约束满足问题

祝恩
计算机学院
国防科技大学



学习内容

1. 定义约束满足问题:

CSP形式化、约束图、约束类型

2. 约束传播(CSP中的推理):

结点相容、弧相容、路径相容、K相容

3. CSP的回溯搜索

4. 提高回溯搜索效率:

变量顺序、值的顺序、提前检查失败(前向检验、弧相容维护)

5. CSP的局部搜索

6. CSP的问题结构

1 定义约束满足问题

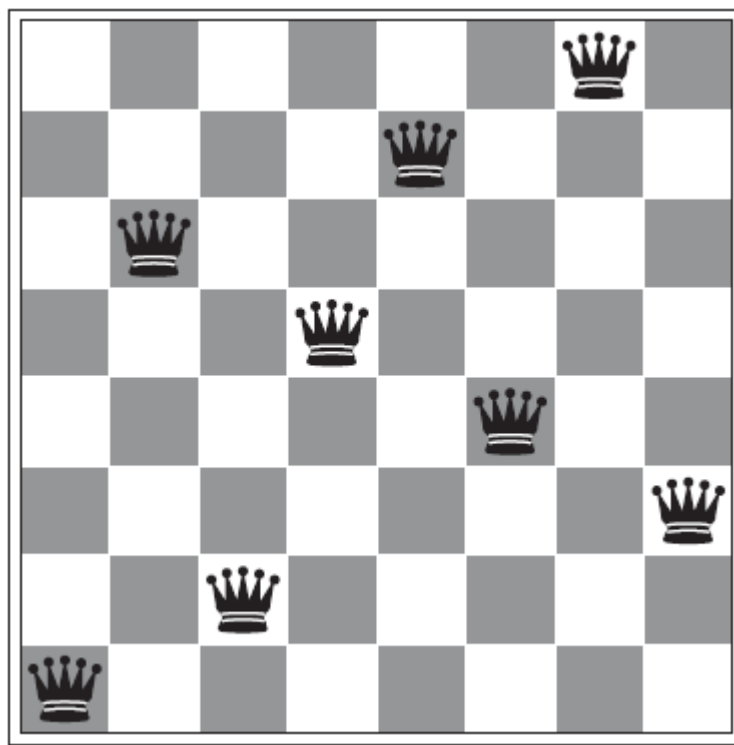


求解n-queens、CSP形式化、约束类型、约束图、约束超图

回顾n-queens的局部搜索

- h = number of pairs of queens that are attacking each other, either directly or indirectly

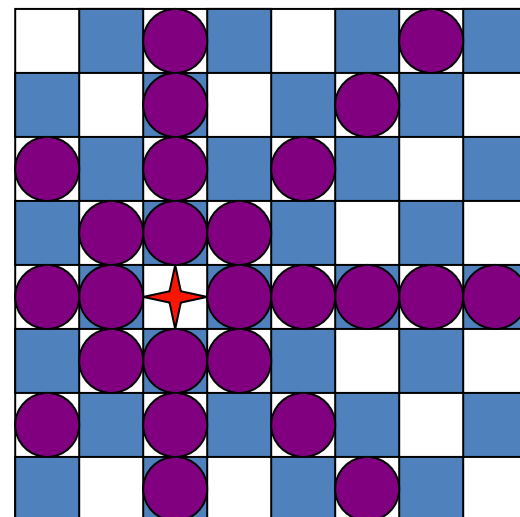
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18



约束传播

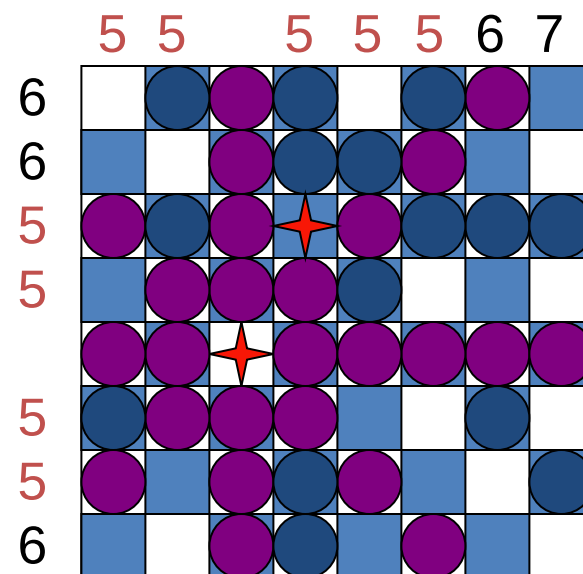
Constraint Propagation

- Place a queen in a square
- Remove the attacked square from future consideration



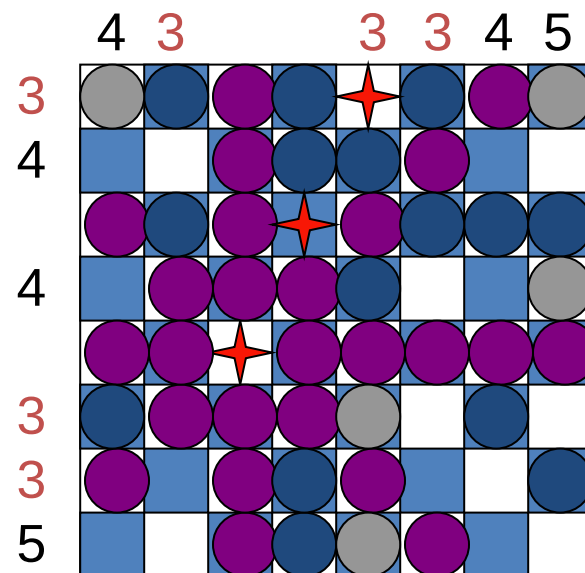
约束传播

- Count the number of non-attacked squares in every row and column
- Place a queen in a row or column with minimum number
- Remove the attacked squares from future consideration



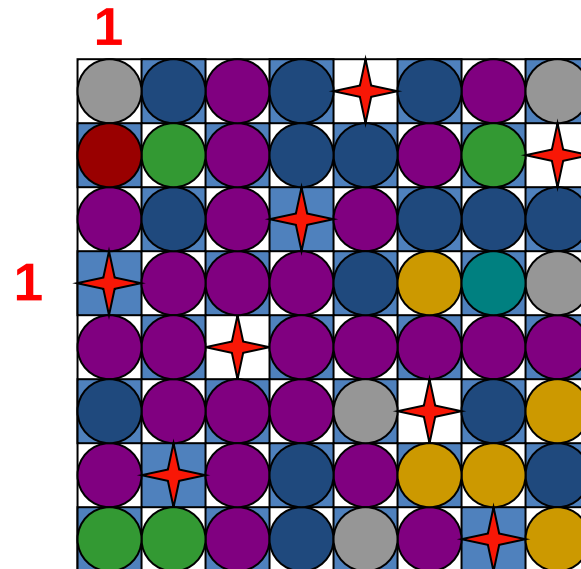
约束传播

- Count the number of non-attacked squares in every row and column
- Place a queen in a row or column with minimum number
- Remove the attacked squares from future consideration



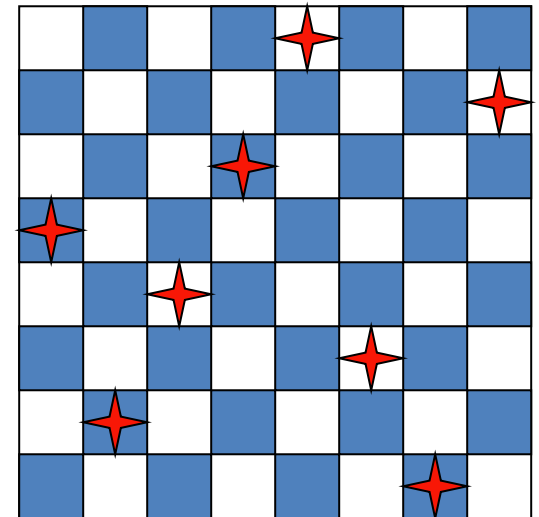
约束传播

- Count the number of non-attacked squares in every row and column
- Place a queen in a row or column with minimum number
- Remove the attacked squares from future consideration



约束传播

- Count the number of non-attacked squares in every row and column
- Place a queen in a row or column with minimum number
- Remove the attacked squares from future consideration



CSP的形式化

□ a CSP consists of

■ a set of variables, $X = \{X_1, \dots, X_n\}$.

■ a set of domains, $D = \{D_1, \dots, D_n\}$.

■ a set of constraints, C .

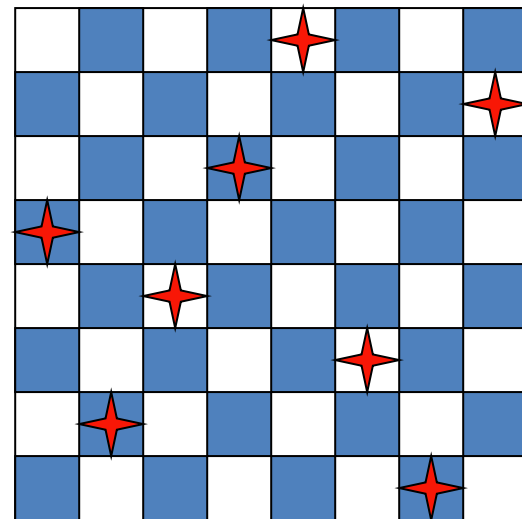
Each constraint C_i consists of $\langle \text{scope}, \text{rel} \rangle$.

scope is a tuple of variables, and *rel* defines the relation between those variables.

□ Solutions are complete and consistent assignments

8-queens的CSP形式化

- CSP formulation of 8-queens
- Variables: $X = \{X_1, X_2, X_3, \dots, X_8\}$
- Domain: $D = \{D_1, D_2, D_3, \dots, D_8\}$
 $D_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$
- Constraints: $C = \{ \langle \{X_i, X_j\}, X_i \neq X_j \ \& \ |X_i - X_j| \neq |i - j| \rangle \mid i \neq j, 1 \leq i \leq 8, 1 \leq j \leq 8 \}$
- a solution $\{5, 2, 4, 6, 8, 3, 1, 7\}$



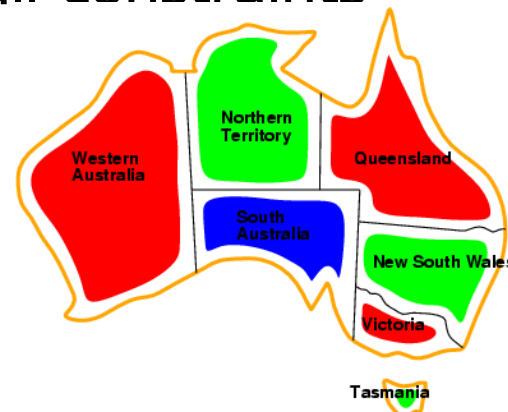
地图着色的CSP形式化

- Variables: $X = \{WA, NT, Q, NSW, V, SA, T\}$
- Domain: $D = \{D_1, \dots, D_7\}, D_i = \{red, green, blue\}$
- Constraints: adjacent regions must have different colors
 $C = \{WA \neq NT, WA \neq SA, NT \neq SA, NT \neq Q, SA \neq Q, SA \neq NSW, SA \neq V, Q \neq NSW, NSW \neq V\}$

$SA \neq WA$ 是 $\langle (SA, WA), SA \neq WA \rangle$ 的简洁表示

- Solutions are assignments satisfying all constraints

$\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$



约束的类型

- **Unary** constraints involve a single variable:

$$SA \neq green$$

- **Binary** constraints involve pairs of variables:

$$SA \neq WA$$

- **Higher-order** constraints involve 3 or more variables. `between(x,y,z)`

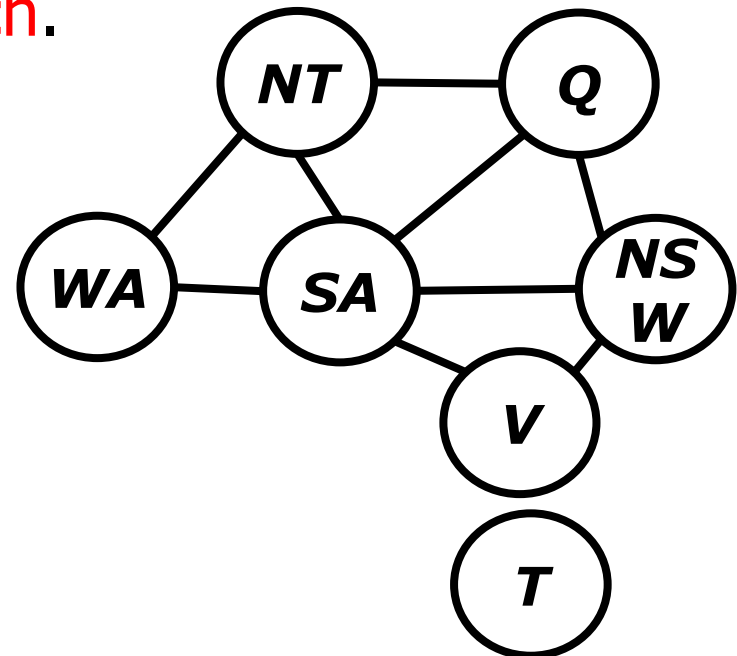
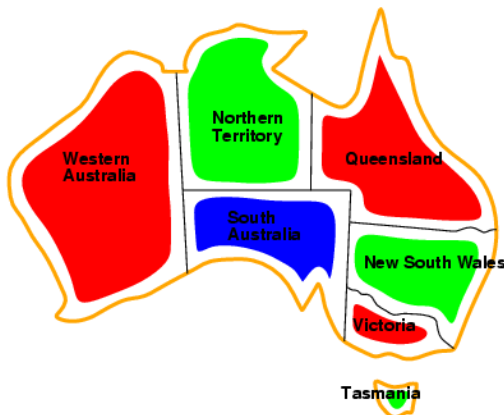
- **Global constraints** involve an arbitrary number of variables. `Alldiff(x,y,z,u)`

- **Preferences** (soft constraints)

- red is better than green
- Gives constrained optimization problems
- Representable by a cost for each variable assignment

约束图

- **Binary CSP**: each constraint relates (at most) two variables
- Constraint graph: **nodes are variables, arcs show constraints**
- General-purpose CSP algorithms **use the graph structure to speed up search.**

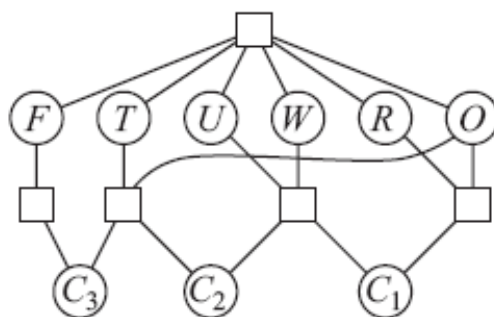


约束超图

- 变量个数任意的约束称为**全局约束**，但不一定涉及所有变量。例如Alldiff
- **约束超图**：表示高阶约束

$$\begin{array}{cccc} T & W & O & \\ + & T & W & O \\ \hline F & O & U & R \end{array}$$

(a)



(b)



写出该问题的约束

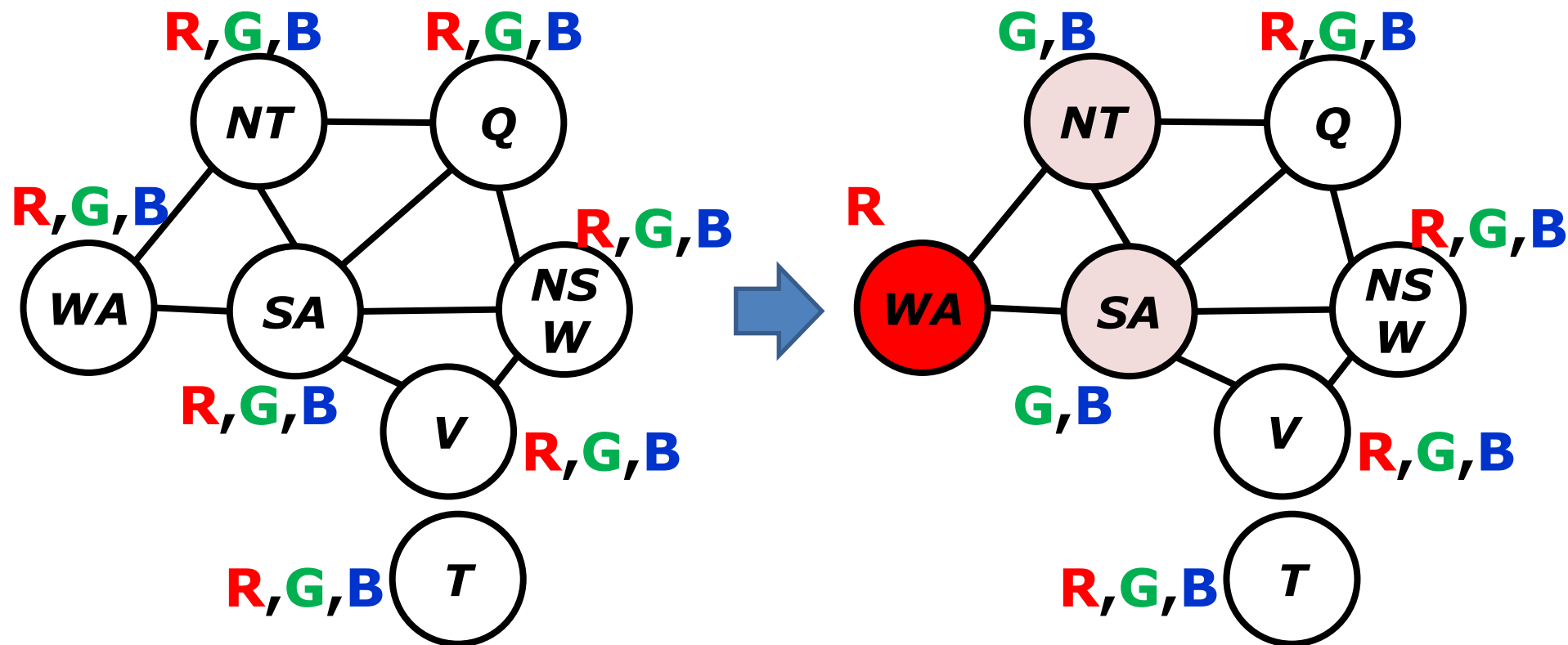
2 约束传播：CSP中的推理



结点相容(1-相容)、弧相容(2-相容)、路径相容(3-相容)、K-相容

约束传播

□ 若WA赋值R，则通过约束传播，NT和SA的值域中删除R

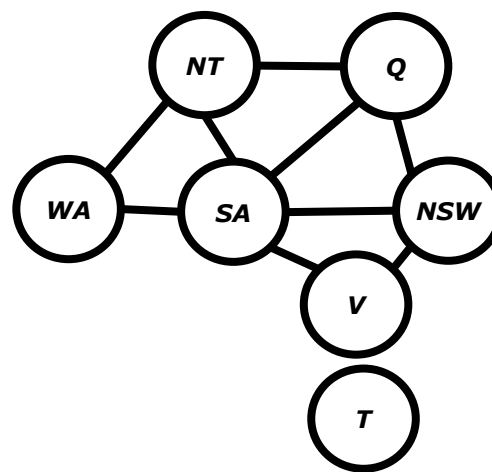


约束传播和搜索

- 在CSP中,算法可以进行约束传播,也可以搜索 (从几种可能性中选择新的变量赋值)。约束传播与搜索可以交替进行,也可以将约束传播作为搜索前的预处理步骤。

局部相容性

- **约束传播的核心思想是局部相容性：**增强约束图中各部分局部相容性会导致不相容的结点取值被删除。



结点相容

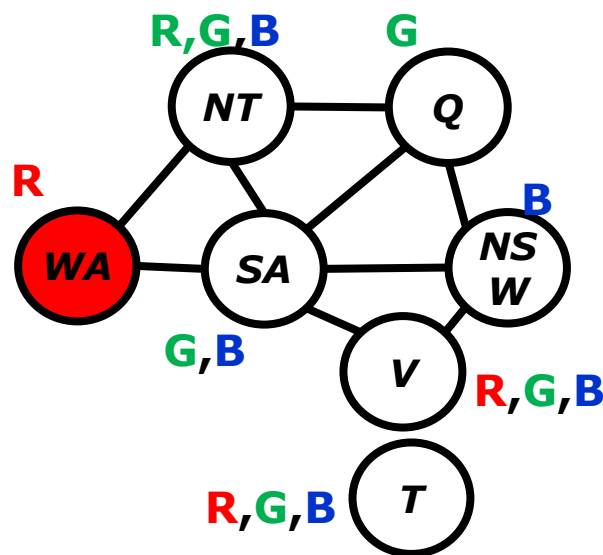
Node consistency

- 如果单个变量（对应于CSP网络中的结点）值域中的所有取值满足它的一元约束，就称此变量是结点相容的。
 - 例如，如果地图着色问题中南澳洲人不喜欢绿色，变量 SA 原来值域为 $\{red, green, blue\}$ ，删除 $green$ 此结点即为结点相容的，此时 SA 的值域为 $\{red, blue\}$ 。
- 如果网络中每个变量都是结点相容的，则此网络是结点相容的。

弧相容

arc consistency

- 若对 X_i 的每个赋值, X_j 都存在某个取值满足弧 (X_i, X_j) 的二元约束, 则称 X_i 关于 X_j 是弧相容的。



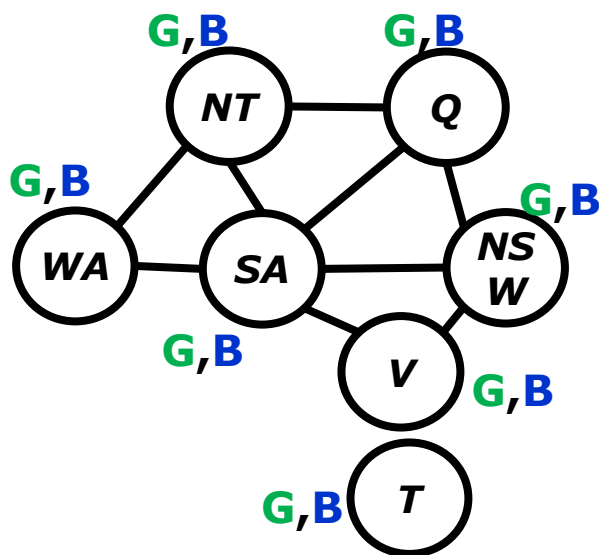
哪些是弧相容的?

- (1) WA关于SA
- (2) WA关于NT
- (3) SA关于Q
- (4) SA关于NSW

弧相容

arc consistency

- 若对 X_i 的每个赋值, X_j 都存在某个取值满足弧 (X_i, X_j) 的二元约束, 则称 X_i 关于 X_j 是弧相容的。
- 如果每个变量关于其它变量都是弧相容的, 则称该网络是弧相容的



网络是否是弧相容的?
是否有解?

弧相容算法AC-3

function AC-3(csp) **returns** false if an inconsistency is found and true otherwise

inputs: csp , a binary CSP with components (X, D, C)

local variables: $queue$, a queue of arcs, initially all the arcs in csp

while $queue$ is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(queue)$

if REVISE(csp, X_i, X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

add (X_k, X_i) to $queue$

return true

去掉
- $\{x_j\}$

function REVISE(csp, X_i, X_j) **returns** true iff we revise the domain of X_i

$revised \leftarrow false$

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

delete x from D_i

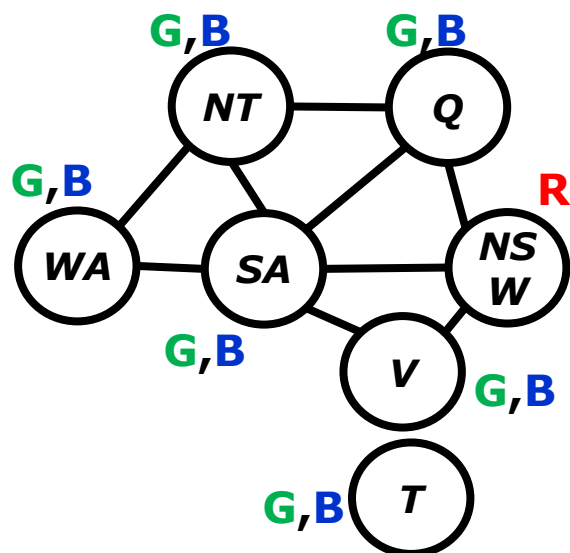
$revised \leftarrow true$

return $revised$

路径相容

path consistency

- 对 $\{X_i, X_j\}$ 的每一个相容赋值 $\{X_i=a, X_j=b\}$, X_m 都有合适的取值同时使得 $\{X_i, X_m\}$ 和 $\{X_m, X_j\}$ 是相容的, 则称集合 $\{X_i, X_j\}$ 关于 X_m 是路径相容的。



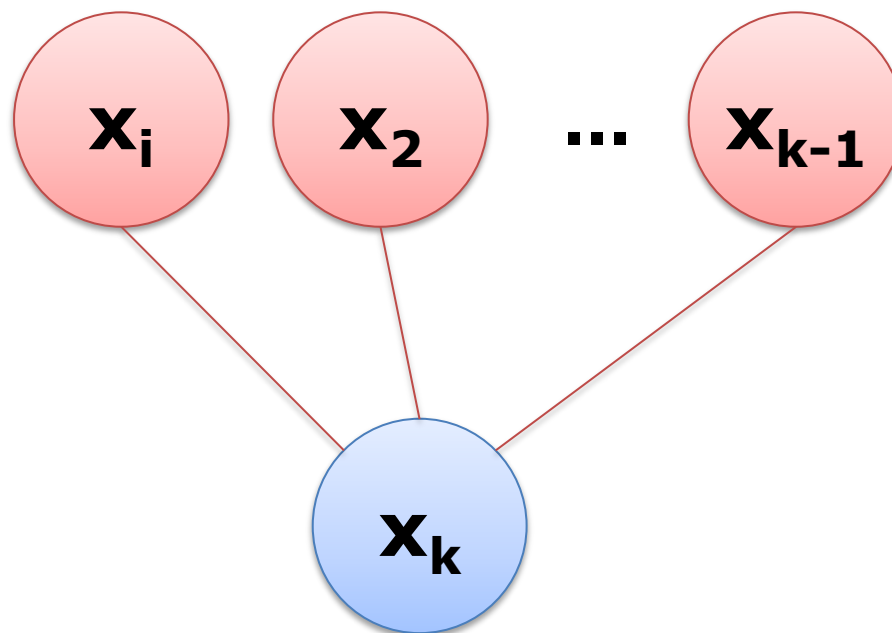
哪些是路径相容的?

- (1) $\{WA, NT\}$ 关于 SA
- (2) $\{SA, Q\}$ 关于 NSW
- (3) $\{SA, NSW\}$ 关于 V

k相容

k-consistency

- 给定k个变量 $\{x_1, \dots, x_k\}$, 对于 $\{x_1, \dots, x_{k-1}\}$ 的每一个相容赋值, x_k 都有合适的取值使得 $\{x_1, x_k\}, \dots, \{x_{k-1}, x_k\}$ 是相容的, 则称 $\{x_1, \dots, x_{k-1}\}$ 关于 x_k 是k相容的。



k相容

k-consistency

- 如果对于任何 $k - 1$ 个变量的任何相容赋值，任何第 k 个变量总能被赋予一个和前 $k - 1$ 个变量相容的值，那么称这个CSP是 k 相容的。
- 1-相容是节点相容。
2-相容是为弧相容。
对二元约束网络，3-相容是路径相容。

全局约束的相容性检查

- **全局约束**可涉及任意个约束变量（不一定是所有变量）。
- **Alldiff约束**表示所有相关变量必须取不同的值。
Alldiff约束的不相容检测的一种简单形式：如果约束涉及 m 个变量，可能的不同取值有 n 个，且 $m > n$ ，那么约束不可能满足。
- **资源约束**，有时称为**atmost约束**。例如在调度问题中，用 $P_1, \dots, P_4$ 表示执行四项任务的人数。总人数不超过10人的约束记为 $atmost(10, P_1, P_2, P_3, P_4)$ ：检测值域的最小值之和可以检测相容性。

3 CSP的回溯搜索



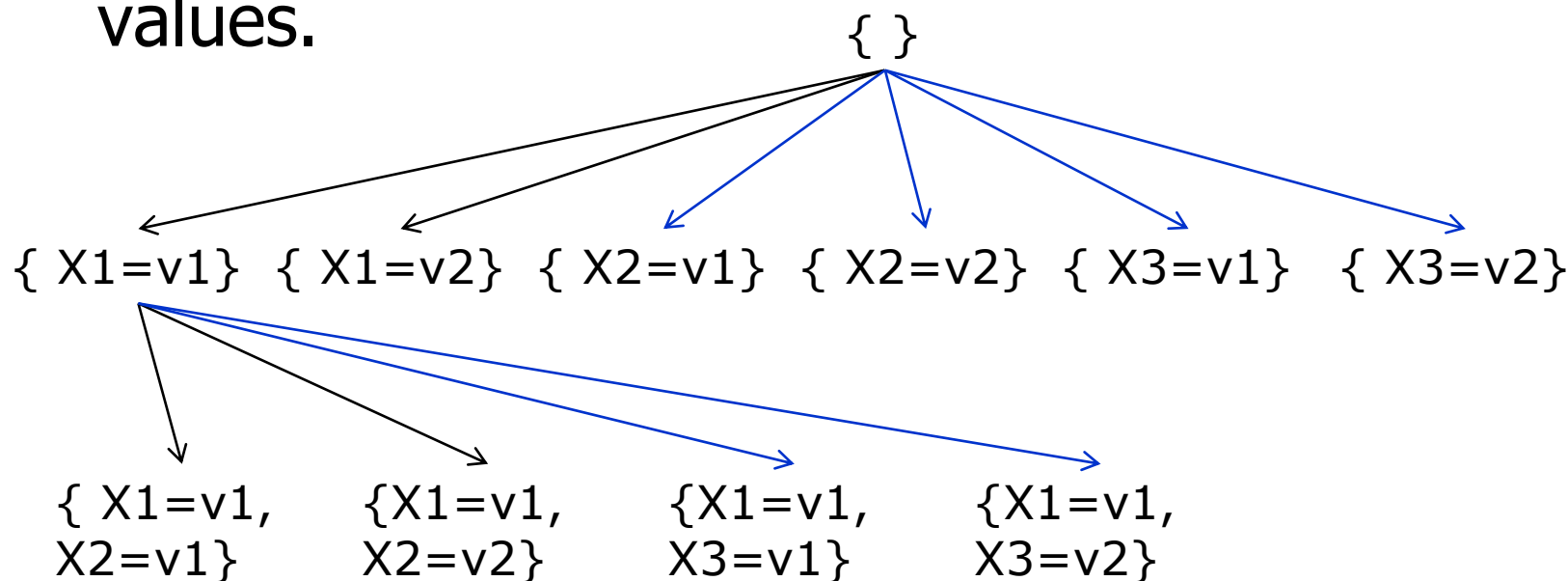
形式化为一个搜索问题，对其进行改进

形式化为一个标准搜索

- States are defined by the values assigned so far
- Initial state: the empty assignment, $\{\}$
- Transition model
 - assign a value to an unassigned variable
 - fail if no legal assignment
- Goal test: the current assignment is complete and satisfies all constraints
- Path Cost
- Actions(s)

ACTIONS(s)

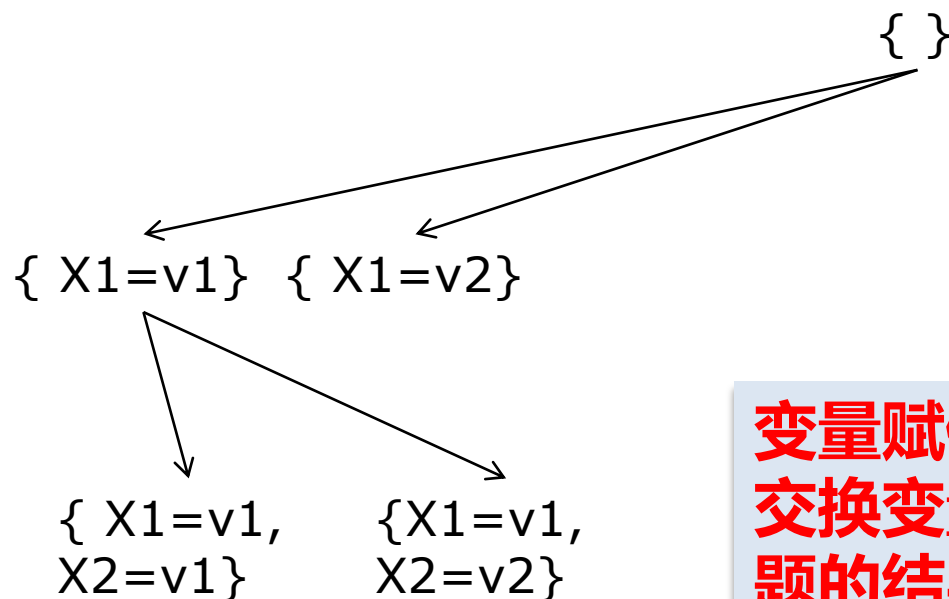
- Suppose there are 3 variables, and each has 2 values.



Actions(s)中动作的数量等于未赋值的变量数
* 平均每个变量的值域大小。如何设法减小分支
因子——减少**Actions(s)**中动作的数量？

每步只考虑单个变量

- suppose there are 3 variables, and each has 2 values.



变量赋值的顺序是可以交换的。交换变量赋值的顺序不改变问题的结果。搜索树每一个结点只要考虑一个变量。

回溯搜索

Backtracking Search

- Idea 1: Only consider a **single variable** at each point:
 - Variable assignments are **commutative**
[WA = red then NT = green] same as
[NT = green then WA = red]
 - Only need to consider assignments to a single variable at each step
- Idea 2: Only allow **legal assignments** at each point
 - consider only values which do not conflict previous assignments, might have to do some computation
- Depth-first search for CSPs with these two improvements is called **backtracking search**

回溯搜索算法

function BACKTRACKING-SEARCH(csp) **returns** a solution, or failure
 return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) **returns** a solution, or failure
 if assignment is complete **then return** assignment
 var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(csp)
 for each value **in** ORDER-DOMAIN-VALUES(var, assignment, csp) **do**
 if value is consistent with assignment **then**
 add {var = value} to assignment
 inferences $\leftarrow$ INFERENCE(csp, var, value)
 if inferences $\neq$ failure **then**
 add inferences to assignment
 result $\leftarrow$ BACKTRACK(assignment, csp)
 if result $\neq$ failure **then**
 return result
 remove {var = value} and inferences from assignment
 return failure

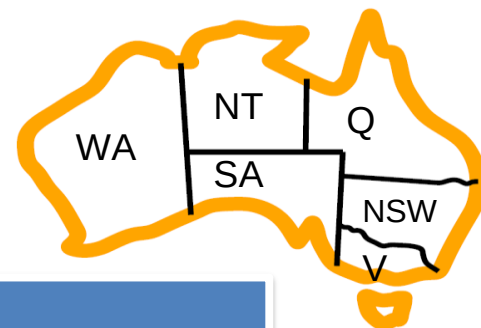
4 提高回溯搜索效率



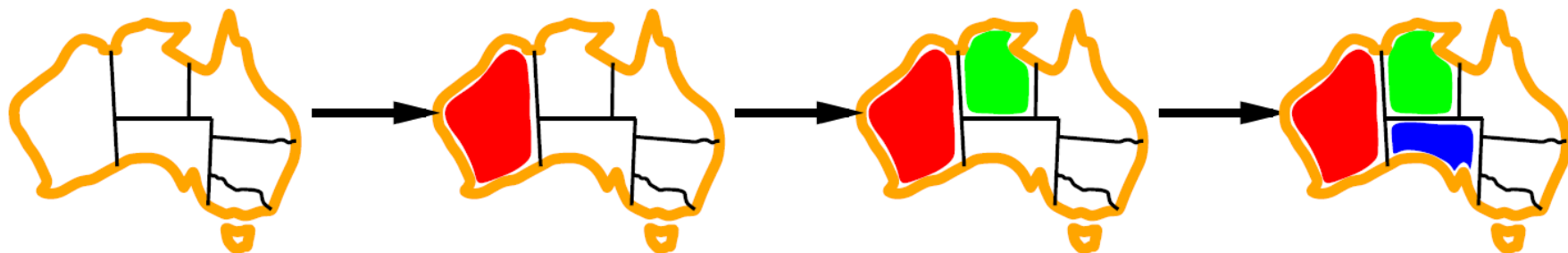
变量顺序（最少剩余值变量优先、最大度变量优先）、
值的顺序（最少约束值优先）、
提前检查失败/如何剪枝（前向检验、维护弧相容）

变量的次序：最少剩余值变量优先

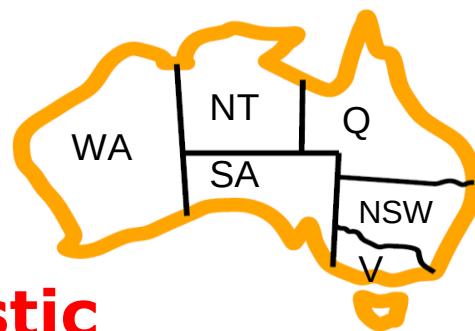
Minimum remaining values



- **最少剩余值(MRV)变量优先：**
优先选择剩余值数量最少的变量。
- 选择剩余值数量多的变量容易使得剩余值数量少的变量的值域变为空值域。
- 也称为“失败优先”启发式，剩余值少的变量的值域容易变为空而失败。

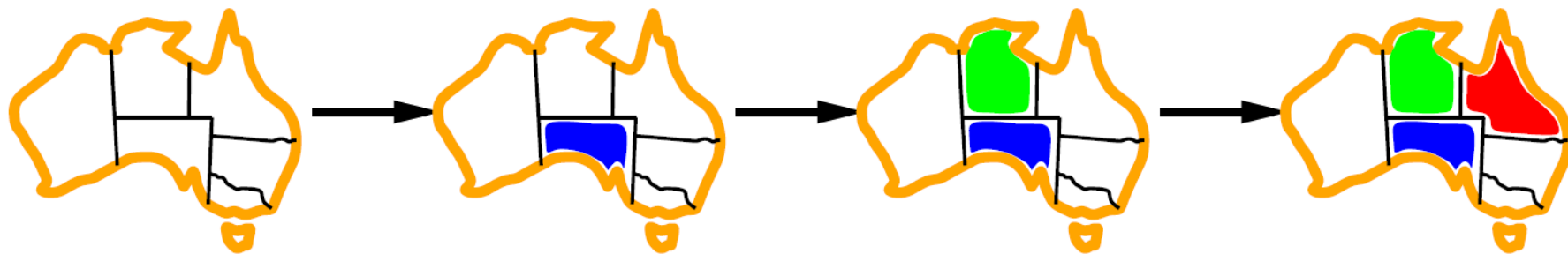


如果剩余值数量相同：最大度变量优先



degree heuristic

- 最大度变量优先：优先选择对其他变量约束最多的变量（约束最多其他变量的变量）。
- 通过选择与其它未赋值变量约束最多的变量来试图降低未来的分支因子

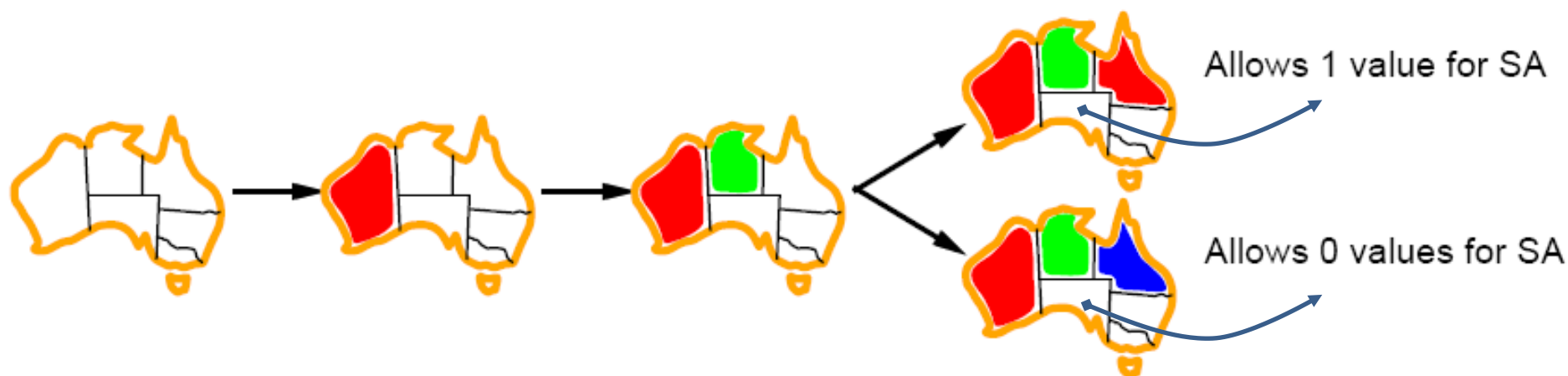


值的次序：最少约束值优先

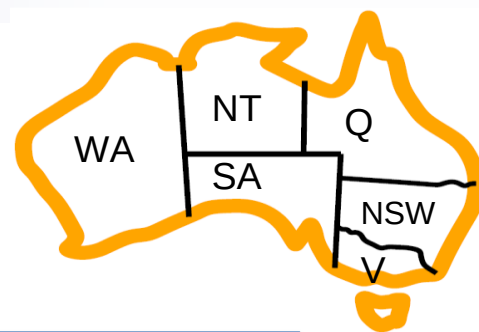
least constraining value



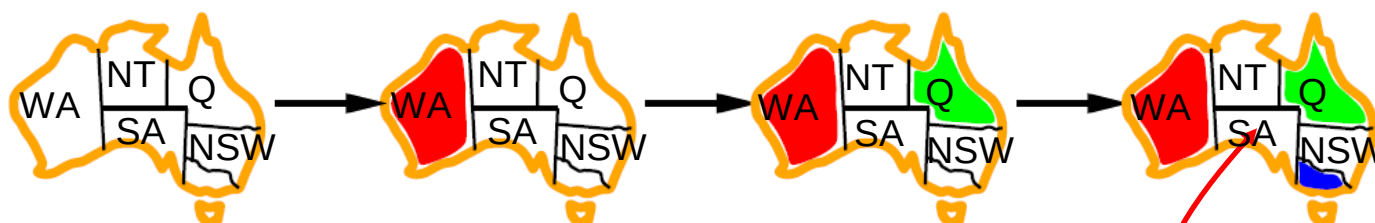
最少约束值优先：选择的取值会删除邻居变量的一些有效值，选择的取值应尽可能少地删除邻居变量的有效值（让它们保留更多的剩余值）。



提前发现失败：前向检验



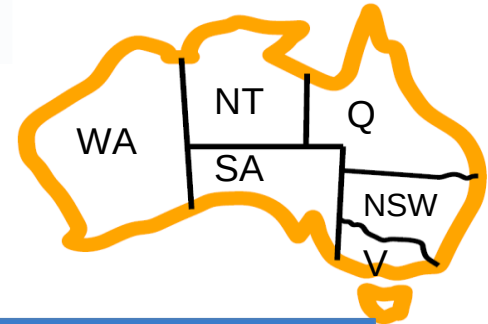
前向检验(Forward Checking): 一个变量X赋值后, 对每个相邻变量Y根据弧相容推理从值域删除与X不相容的值, 如果某个变量值域变为空, 就回溯。



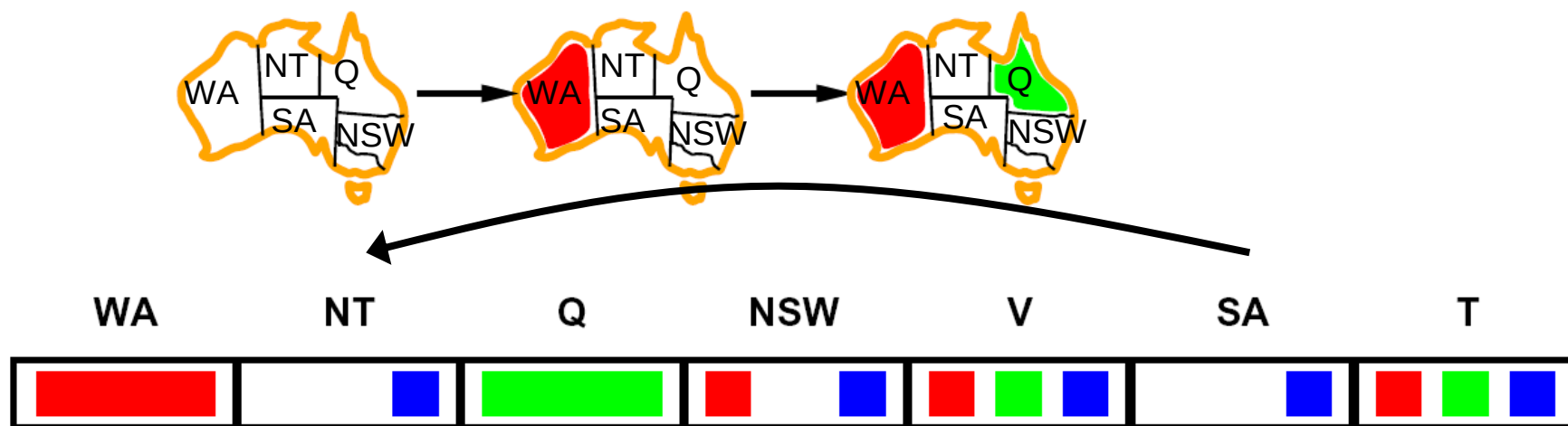
前向检验

WA	NT	Q	NSW	V	SA	T

更早地检测失败：维护弧相容



维护弧相容(MAC): 当变量X赋值后, INFERENCE调用AC-3, 从与X相邻的所有未赋值变量开始, 进行约束传播, 一旦某个变量的值域变为空, 则AC-3调用失败并立即回溯



5 CSP的局部搜索



CSP的局部搜索算法;
逃离高原的方法;
问题改变时局部搜索的优势

CSP的局部搜索

- 局部搜索算法（见第4.1节）对求解许多CSP都是很有有效的。它们使用完整状态的形式化：初始状态是给每个变量都赋一个值，搜索过程是一次改变一个变量的取值。



为一个变量的取新值时，
选择什么样的新值

回溯搜索使用部分赋值的形式化：
初始状态是无变量赋值的状态，搜索过程是**一次选择一个变量赋值。**

CSP的局部搜索算法

- 在为变量选择新值的时候，最明显的启发式是选择与其它变量冲突最少的值——**最少冲突启发式**。

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or failure

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*

for $i = 1$ to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES

value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*var*, *v*, *current*, *csp*)

 set *var* = *value* in *current*

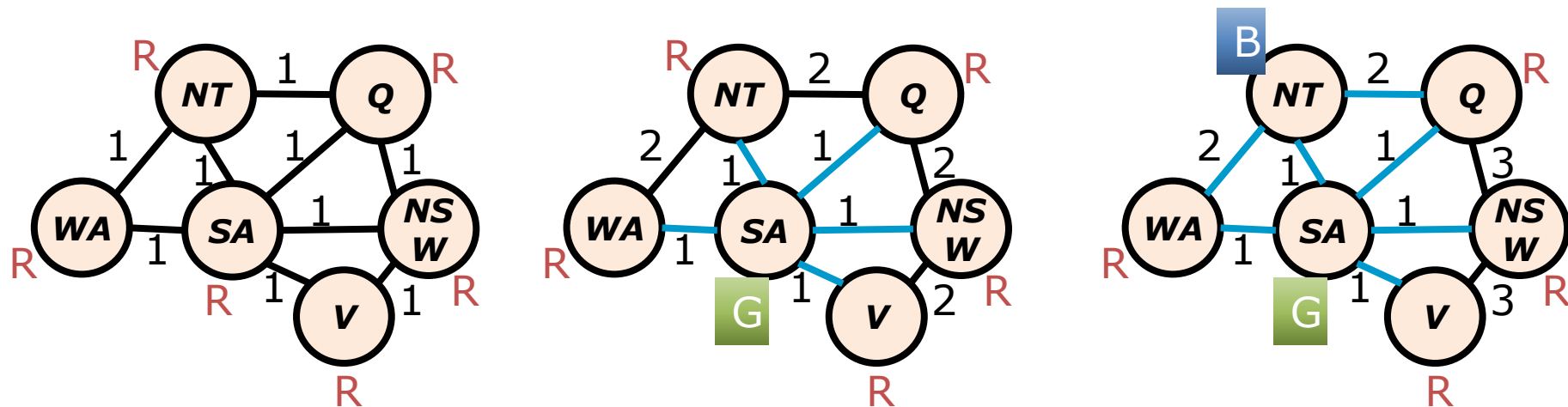
return *failure*

如何逃离高原：禁忌搜索和模拟退火

- “局部搜索(ch04)”中的所有局部搜索技术都是CSP应用的候选，有些技术已经被证实非常有效
- 使用**最少冲突启发式**的CSP的地形通常有一系列**高原**。可能有上百万种赋值，它们都只有一个冲突。
 - **高原搜索**。这可以由**禁忌搜索**引导：将最近访问过的状态记录在表中，并禁止算法再回到那些状态。
 - **模拟退火**也可以用于逃离高原。

如何逃离高原：约束加权

- 另一项技术称为**约束加权**。每个约束都有一个数字权重 W_i ，初始都为1。搜索的每一步，算法都**选择修改一个变量的值，使得违反的约束权重和最小化**。接着增加当前赋值违反的约束的**权重值**。这给高原增加了地形，并且它随着时间的进行不断给难于解决的约束增加权重。



问题改变时局部搜索更容易

- 局部搜索的另一个优势是当问题改变时它可以用于联机设置。这在调度问题中特别重要。一周的航班日程表可能涉及上千航班和上万人员的安排，但是一个恶劣天气可能就会打乱原来的机场日程安排。我们希望在最小的改动来修正日程。
 - a. 局部搜索？
 - b. 回溯搜索？

6 CSP的问题结构



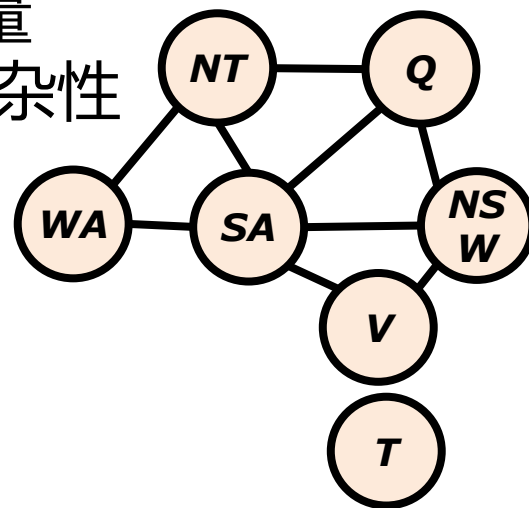
独立子问题

树形结构的CSP的求解算法

化简为树形结构的方法：删除结点(环割集)；合并结点(树分解)

独立的子问题

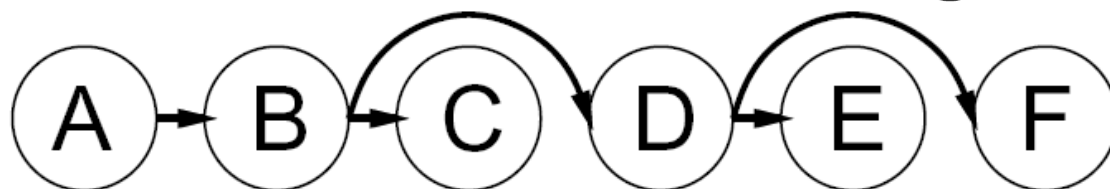
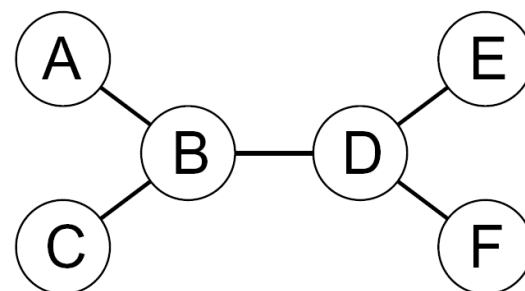
- 独立的子问题 $CSP_1, CSP_2, \dots$
- 独立性通过寻找连通子图确定
- 如果赋值 S_i 是 CSP 的一个解, 那么 $\cup_i S_i$ 是 $\cup_i CSP$ 的一个解。
- 假设变量共 n 个, 每个子问题 c 个变量
一共 n/c 个子问题, 每个子问题的复杂性
为 $O(d^c)$, 总工作量为 $O(d^{cn/c})$
- 不分解的情况下为 $O(d^n)$
- 实例: $n=80, c=20, d=2$
 - $O(d^{cn/c})$: $2^{20 \times 4}$
 - $O(d^n)$: 2^{80}



两个独立的子问题

树形结构的CSP

1. Choose a variable as root, **order variables from root to leaves** such that every node's parent precedes it in the ordering



2. For $i = n : 2$, Make-Arc-Consistent(Parent(X_i), X_i)
3. For $i = 1 : n$, assign X_i consistently with Parent(X_i)

□ Runtime: $O(\rule{1cm}{0.4pt})$

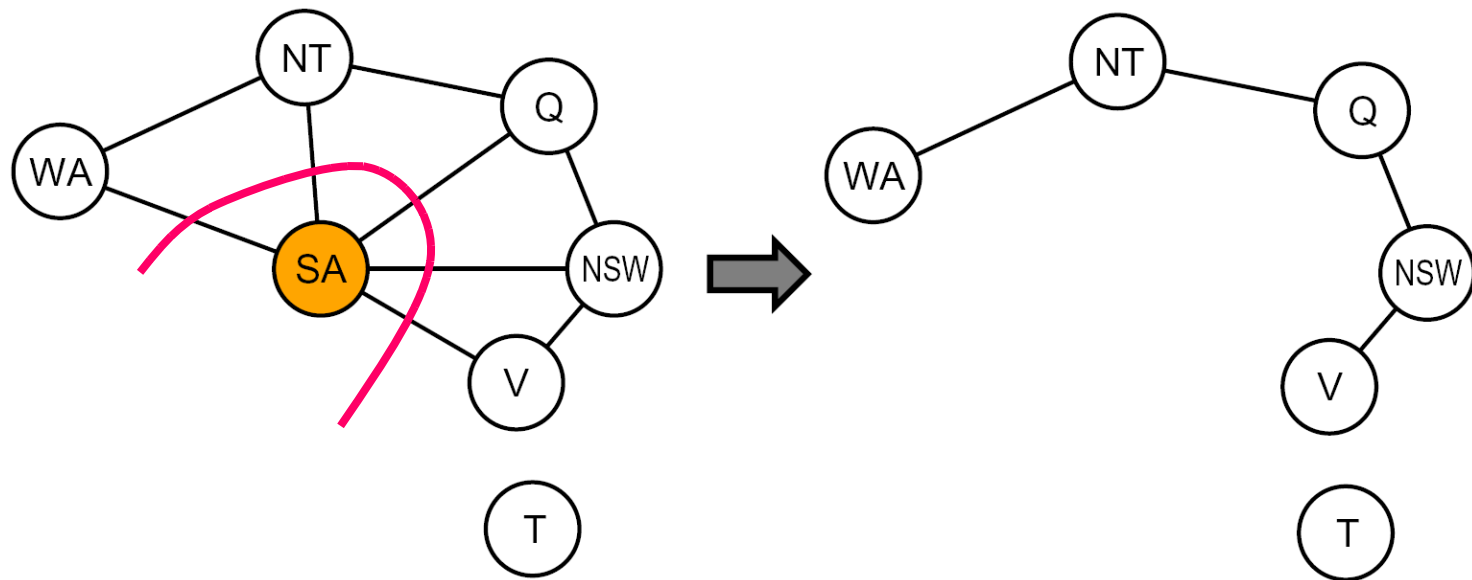


如果不是树形结构的约束图
如何化简为树的形式？



基于删除节点的方法

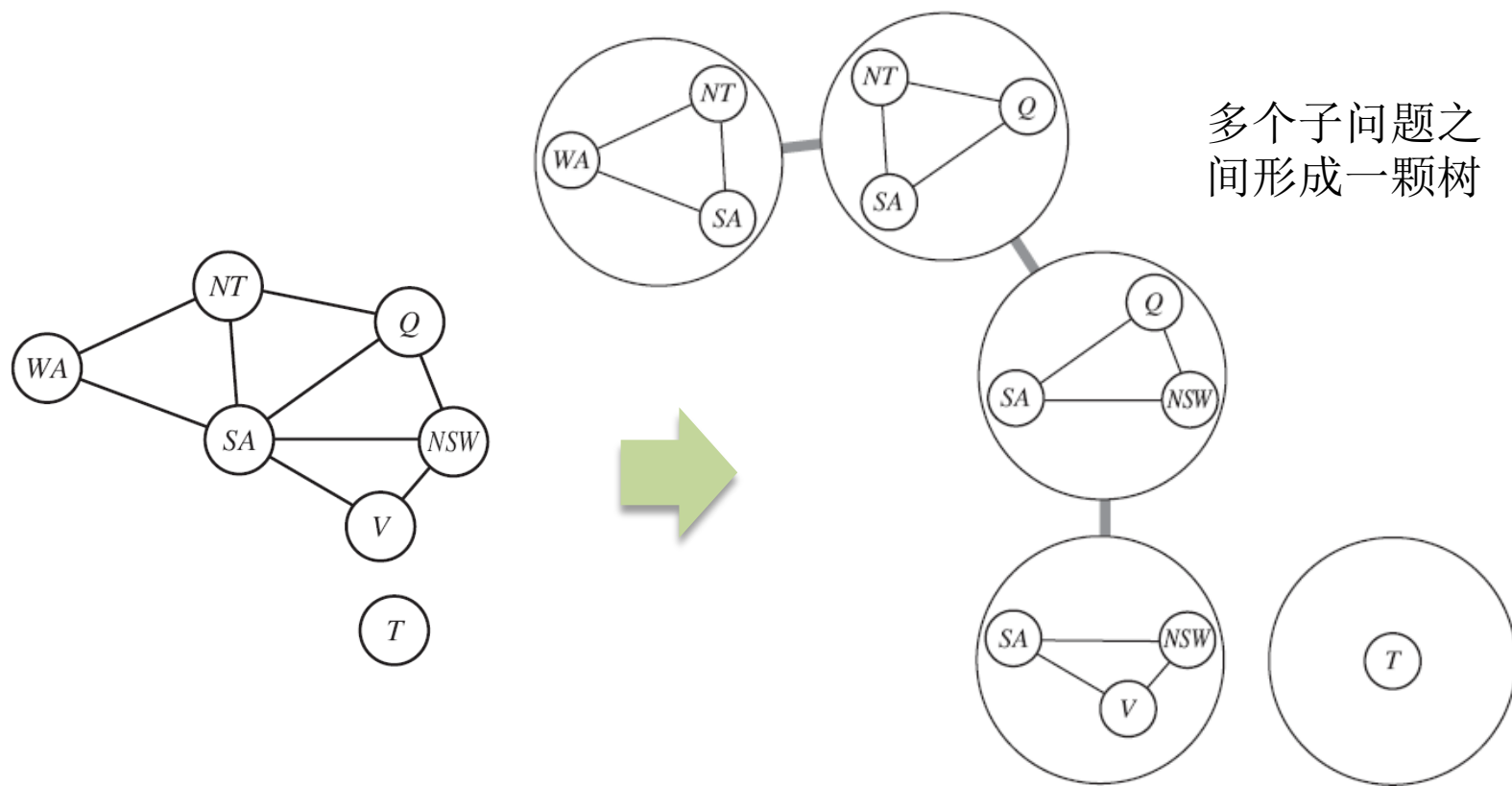
- 先对部分变量赋值，使剩下的变量能形成一颗树
 - 例如给SA赋一个固定的值，修剪其他变量的值域
 - 其他变量的CSP的每个解(用树形结构的算法求解)都与SA的值相容
 - 其他变量的CSP可能无解(此实例中是有解的)，这时需要尝试SA的其他值



(Finding the minimum cutset is NP-complete.)

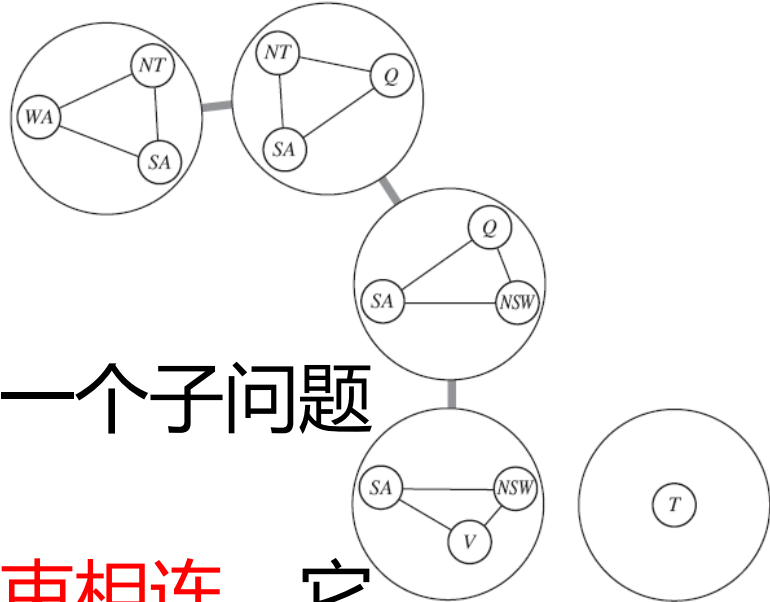
基于合并结点的方法：树分解

- **树分解**将约束图分解为**相关联**的子问题，每个子问题独立求解，再把得到的结果合并起来。
 - 例如，地图着色问题分解为5个相关联的子问题



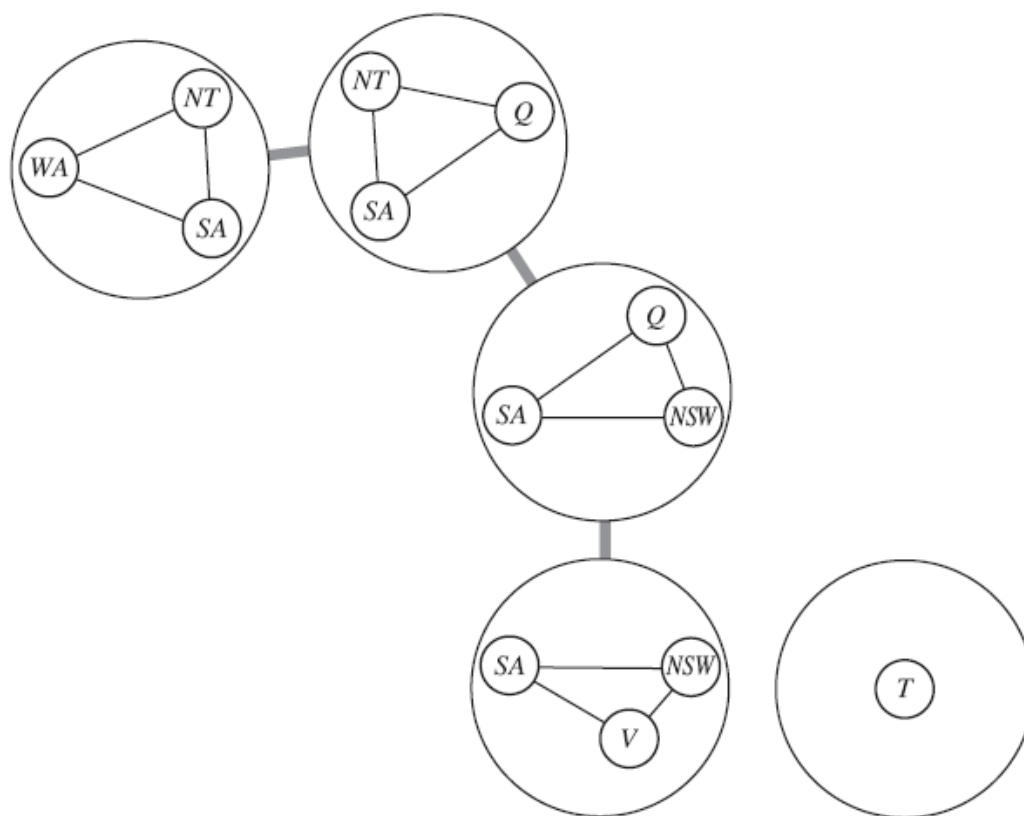
树分解必须满足的条件

1. 原问题中的**每个变量**至少在一个子问题中出现
2. 原问题中的**两个变量若有约束相连**，它们至少同时出现在一个子问题中（连同它们的约束）
3. **如果一个变量出现在树中的两个子问题中**，那么它必须出现在连接这两个子问题的**路径上**的所有子问题里。

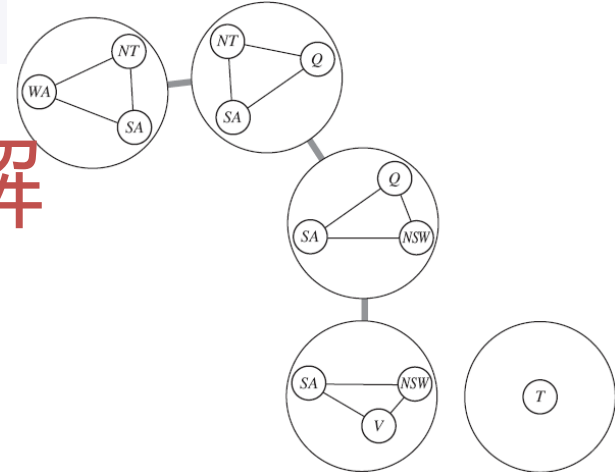


树分解CSP的求解

- 独立求解每个子问题；任何一个无解，整个问题就无解。求解所有的子问题后，设法构造一个完整解。



树分解CSP的求解



- 每个子问题被视为一个“巨型变量”，它的值域是这个子问题的所有解的集合。
 - 例如，图中最左边的子问题的一个解是 $\{WA = red, SA = blue, NT = green\}$ 。
- 用前面给出的树算法来求解连接这些子问题的约束——它们的共享变量要取相同的值
 - 例如，给定第一个子问题的解 $\{WA = red, SA = blue, NT = green\}$ ，下一个子问题的相容解只能是 $\{SA = blue, NT = green, Q = red\}$ 。

Thanks!
